MovIn

Kate Zraly
Computer Systems Technology
British Columbia Institute of Technology
Vancouver, Canada
kzraly@my.bcit.com

Cameron Walford
Computer Systems Technology
British Columbia Institute of Technology
Vancouver, Canada
cwalford@my.bcit.com

Justin Viacrusis
Computer Systems Technology
British Columbia Institute of Technology
Vancouver, Canada
jviacrusis1@my.bcit.com

Ankit Ahlwat
Computer Systems Technology
British Columbia Institute of Technology
Vancouver, Canada
aahlwat@my.bcit.com

Abstract—Exercise-based games like Just Dance and Kinect Sports allow players to get active while being immersed in fun gameplay. However, there are a limited number of games that a user can choose from. The exercise software is bound to only Kinect games, so it isn't possible to incorporate exercise into PC games or console games. Existing studies have used computer vision to classify human actions. We are looking to extend these studies by using human action classification to promote exercise. We will map exercises to keystrokes, so that users can add exercise into any video game. This paper will talk about data visualization, model training and model testing. Then, we will cover the results before concluding and reflecting on future development paths.

I. Introduction

A. Problem Context

Some people struggle to consistently exercise, often due to lack of enjoyment. To motivate people, you need to bring fun into the exercise. A great way to do this is to incorporate exercise into your favorite hobbies. An example is through sports, or dance. Video games are a popular hobby, and would present an opportunity for those who are passionate about games to incorporate exercise into them. Exercise based gaming isn't a novel concept, with technology like the Xbox Kinect successfully motivating players to move and stay active; however, they have their limitations.

Current exercised-based games like Just Dance and Kinect Sports are fun, however they are limited in scope and variety. Many of them are also platform dependent, offer a restricted set of movements, or are no longer even being developed. Furthermore, there is currently no tool that would allow users to integrate fitness into games of their choice, whether it's a puzzle game like 2048 or a role-playing-game like Cyberpunk 2077. Unfortunately this leaves gamers who want to improve their health with very few options.

B. Proposed Solution

Our solution is "MovIn" the movement-to-input system that enable players to incorporate exercise into their existing library of games. Instead of needing to specifically design games for our system, we instead integrate computer vision based

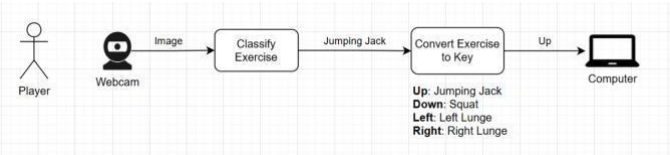

Fig. 1. A high-level overview of MovIn

pose estimation and machine learning classification into the current gaming ecosystem. Our system will recognize a set of movements and the user can freely map these movements to keyboard/mouse inputs, allowing them to control any game they wish, as shown in Fig. 1. The mapping system should be dynamic enough to be applied to any game or computer task, but for the purpose of this project, the main focus will be controlling a game of 2048. This game has four moves: up, down, left, and right. We will map the following exercises to those keystrokes: jumping jacks, squats, left lunge, and right lunge.

This solution allows the user to integrate MovIn into any video game, so that users can enjoy playing PC and console games that are not typically exercise based. This application extends previous human action models with versatile func-tionality. One of the challenges prevalent in human action classification that also applies to this paper is collecting au-thentic and sufficient data. While games dedicated to exercise closely couple computer vision to the game itself, our solution provides an interface to connect your own set of exercise moves to any game. With this exercise to input model, we hope to increase accessibility to exercise for those passionate about video games.

C. Jetson Nano and Edge AI

As we developed this approach, we also considered the practicality of deployment on portable, low-cost computing devices. To explore these constraints, we integrated and tested our model on an NVIDIA Jetson Nano. This Edge AI de-vice forces us to optimize performance to achieve near real-time movement classification under limited computational

resources. By ensuring the model runs effectively on a Jetson Nano, we can pave the way for a wider range of accessible, portable exercise-gaming setups.

D. Pose Classification

Within the field of human action recognition, there are two main methods to classify poses.

1) Image Classification: A commonly used technique to classify poses is image classification. This is when a model, usually a CNN, is trained directly on a large dataset of RGB images as shown in Figure 2.

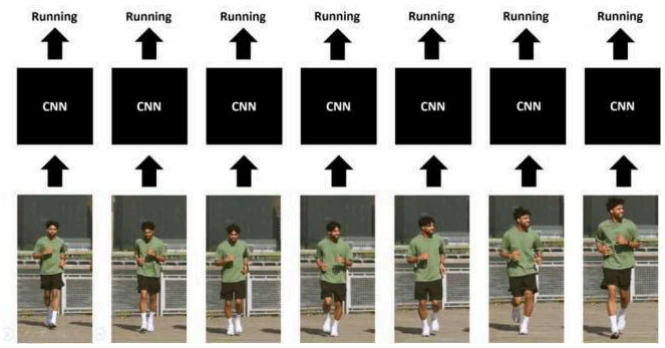

Fig. 2. Movement classification using images [2]

2) Landmark Classification: Another technique to classify poses is landmark classification. This is when the images in a dataset are first preprocessed with a pose estimation model like Google's BlazePose [3] to extract labeled landmarks of key points on the human body as seen in Fig 3. A classifier is then trained on only these landmarks.

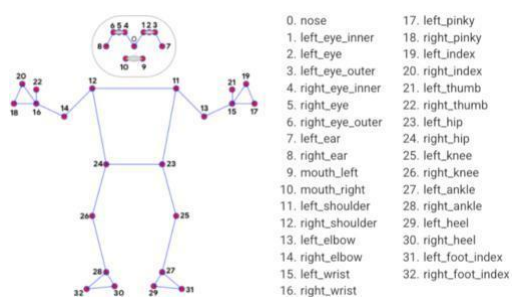

Fig. 3. BlazePose landmarks representing 33 key points on the human body [3]

We chose to use landmark classification for its efficiency, scalability, and potential for higher generalization. Some comparisons are shown in Table I. Focusing on key point coordinates rather than raw image pixels allows us to reduce the model's sensitivity to variations in lighting, background, gender, clothing, and more. Overall, utilizing landmarks allows us to build a much more efficient and robust model with fewer samples.

	Image	Landmarks
Datasets	Many	Few
Input Preprocessing	No	Yes
Dimensionality	High	Low
Training Time	High	Low
Variation Sensitivity	High	Low

TABLE I
COMPARISON OF IMAGE CLASSIFICATION AND BLAZEPOSE LANDMARK CLASSIFICATION.

E. Related Works

Some related works we found include "Gesture Control Gaming" [1], "Human Activity Recognition Using Pose Es-timation" [5], and "Human Action Recognition using 3D Convolutional Neural Networks" [4]. Each touch on areas of our solution.

Gesture Control Gaming [1] maps movements to game controls using a pose estimation model; however, the examples given are restricted to a driving simulator and only rely on movement of the arms.

The research done on action recognition with 3D CNNs was promising, but we had our concerns in it's applicabil-ity for real-time gaming. In the paper they create a model that is trained on a sequence of RGB images of a person performing a particular action [4]. Their results were great, with high f-score; however, the issue lied with the increased dimensionality of the input data. Inference times on images alone can be quite high, so inference times on multiple images simultaneously is understandably much higher.

Gupta et al. [5] propose a well balanced solution: using a sequential approach with pose estimation to extract human body landmarks from a single image and feeding those into a classification model to predict the activity being done in frame. Though, this still lacked the modularity MovIn aims to provide.

F. Paper Roadmap

The rest of the paper is laid out as follows: section II covers data visualization, section III covers methodology, including model training, testing and deployment, section IV covers results, section V covers our conclusion, and section VI covers future work to enhance the capabilities and user experience of MovIn.

II. DATA EXPLORATION

A. Lack of Applicable Movement Datasets

Although human movement classification is not a novel concept, we could not find any datasets that could be applied to our desired use case. The main problem with these datasets was their inherent lack of granularity. For instance, the popular human motion dataset, HMDB51, contains videos for the "punch" movement, but it does not distinguish which arm is being used in the punch, nor the type of punch [Fig 4].

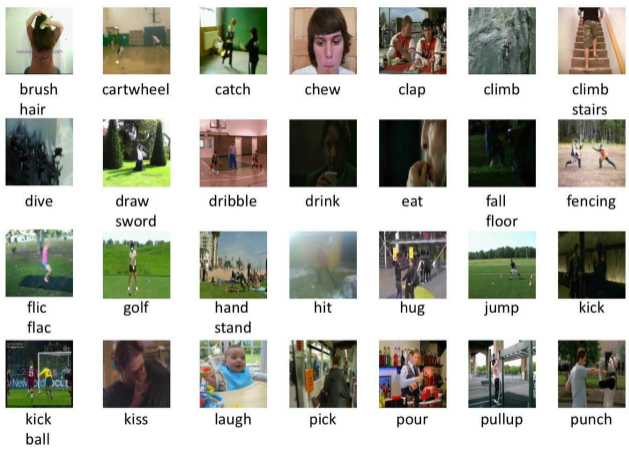

Fig. 4. Examples of categories in the HMDB51 dataset [6]

B. Creating our Data

We didn't have a dataset to work with and were limited in human resources for manually labelling new data, so instead, we developed a data pipeline to give us flexibility in our research and development. The pipeline ingests mp4 files from a directory named after your movement, feeds each frame into BlazePose to generate 3D landmarks, uses those landmarks to determine relevant joint angles, and outputs only the frames that meet the joint angle requirements for the movement in question.

1) Sourcing the Data: Data for each movement was col-lected from YouTube videos and formatted as mp4 files. These videos were then placed in directories named after the movement they were for, allowing our pipeline to distinguish which movement to extract relevant video frames for.

2) Extracting the Data: To know which frames of the input videos were relevant to an exercise, we performed a dot product-based joint angle calculation from the 3D BlazePose landmarks generated for each frame. These calculated joint angles were compared against manually defined joint angle thresholds for each movement; if angles in that frame matched the thresholds, the landmarks for that frame were saved for use in model training.

This solution works because human poses can be described by the collective angles of each joint. Furthermore, only a few main joint angles change significantly during most calisthenic exercises, so we only had to define minimum and maximum thresholds for a few joint angles per movement. For instance, jumping jacks were identified when both shoulder angles (hip-shoulder-elbow) were from 70° to 180° and both hip angles (knee-hip-shoulder) were 100° to 180°.

C. Validating our Data

We created overlay videos for each mp4 processed to validate the landmark data extracted from it. A screenshot of one such overlay video is shown in Fig. 5. Along with the

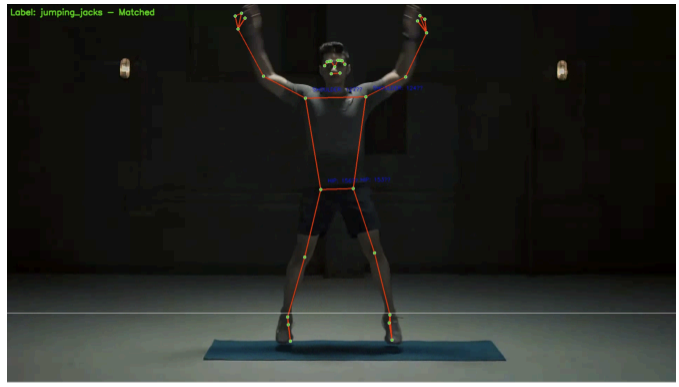

Fig. 5. A frame outputted by our data pipeline for the "jumping jacks" movement. The top left indicates whether the frame matches the movement label, and the numbers in blue indicate the calculated joint angles for relevant joints: right shoulder (127°), left shoulder (124°), right hip (156°), left hip (153°).

landmarks and overlay video, we saved each matching frame. This allowed controlled comparison of individual frames, ensuring we had set joint angle thresholds appropriately.

III. Methods

A. Model Training

Our goal was to choose and train a model that generalizes well to new, unseen exercise videos by recognizing patterns in joint behavior and positions. We chose to use a neural network because neural networks can find relationships between key point coordinates. The neural network will train on samples with landmarks as the data and the type of exercise as the label. It will then be able to predict the type of exercise given only the landmarks. We built the neural network with TensorFlow Keras, defining our own number of layers and the type of layers.

The following table shows the layers of the neural network:

Layer Type	Output Shape	Activation Function
Input Layer	99	none
Dense Layer	128	ReLU
Dropout Layer (30%)	128	none
Dense Layer	64	ReLU
Dropout Layer (30%)	64	no

		softmax
Dense Layer (Output)	num exercises	

TABLE II
LAYER ARCHITECTURE FOR THE NEURAL NETWORK.

The 99 nodes in the input layer correspond to an x, y, and z coordinate for each of the 33 key points. There are two dense layers, and two dropout layers. The dense layers use ReLu for their activation function, which introduces non-linearity. The first dropout layer prevents over-fitting in higher dimensions. The second dropout layer prevents over-fitting as dimensionality decreases and the network starts learning more specific patterns. Finally, the output layer applies Softmax activation to convert the output of the second dropout layer into a set of probabilities for all the exercises. The probabilities sum up to one.

The landmark training data is split into 80% for training, 20% for validation. The neural network is trained in 50 epochs, with a batch size of 32. It utilizes early stopping with a patience of two epochs. This prevents over-fitting by stopping the training process early if the validation loss doesn't improve in 2 consecutive epochs.

B. Application Architecture

We developed an object oriented program for the user to perform exercises in front of a camera and execute those exercises as inputs on the host device with the video game. Figure 6 shows the full pipeline, which gets executed every frame while the program is running.

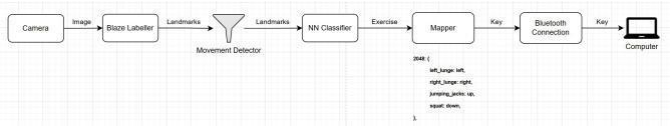

Fig. 6. System workflow: Images are processed to extract landmarks, which are filtered and classified. The classification is mapped to an output action and transmitted via Bluetooth for execution.

The program takes a photo using the host's camera, then sends the image to the Blaze labeller. The labeller extracts the landmarks from the frame. Before classification, the landmark runs through the movement detector. The movement detector acts like a filter which only allows landmarks to pass if the amount of movement is above a certain threshold. If there is enough movement, the landmarks are passed into the neural network, which classifies the corresponding exercise. The exercise is checked against a mapper. This mapper is a class which holds the configuration information from exercise to HID key for each game in the system. When the HID key is found, the mapper sends it over a Bluetooth connection to the target device. The HID key is executed in the game.

C. Bluetooth Integration

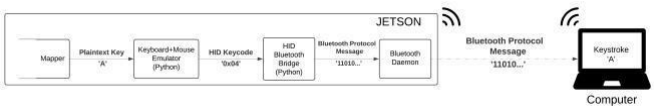

Fig. 7. A diagram of the different components used and data transferred during the Bluetooth device process.

The MoveIn system needed to be able to send keyboard and mouse actions to the user's devices, so different avenues of communication were considered. Bluetooth was chosen due to being supported by the majority of end-user devices, while not requiring additional hardware such as cables or ports. The Jetson runs a modified version of the pre-packaged Ubuntu Bluetooth daemon, and broadcasts as a Bluetooth mouse and keyboard combination device, with the name "jetbot". After a user pairs their device to the Jetson via Bluetooth, a listening python service bridges together the emulated HID Keyboard+Mouse and the modified Bluetooth daemon.

The mapper sends a plaintext string containing keys such as the letter 'A' to the emulated keyboard and mouse device. The key is encoded into an HID keycode and further encoded into a Bluetooth protocol message, before it is transferred to the user's device via Bluetooth connection. The user's device receives this and decodes the message into a keyboard or mouse action. The user's device does not need any additional software or hardware, making it work seamlessly just like any other Bluetooth device.

D. Algorithmic Optimization

The purpose of the movement detector is to reduce the number of calls to the neural network and reduce latency. To do this, the movement detector saves the latest ten frames. As the program continues to capture frames, those new frames are added to the sliding window, and the old ones are discarded (See Figure 8 below, specifically the transition between steps one and two). At every movement detection check, we get a few of the oldest frames from the sliding window and average out their landmarks, then get the last few frames from the sliding window and average out their landmarks. Using the oldest average landmarks and the newest average landmarks, we calculate the Euclidean distance. If the distance is above a certain threshold, the landmarks are sent to the neural network, otherwise they are discarded.

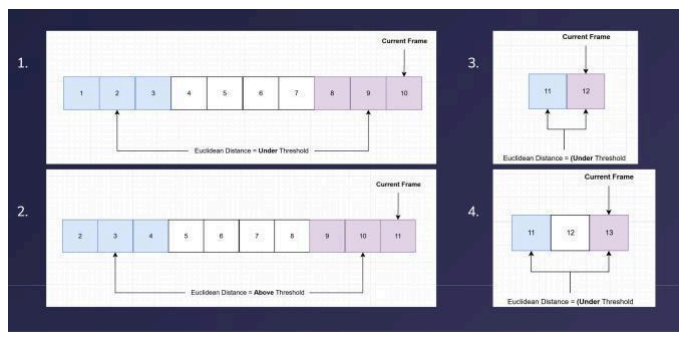

Fig. 8. A diagram of the different components used and data transferred during the Bluetooth device process.

In the diagram in Figure 8, step one shows a frame being added to the sliding window of size 10. The Euclidean distance is calculated, and result is determined to be under the thresh-old. The landmarks are dropped and a new frame is captured in step two. This time, the Euclidean distance lies above the threshold. The landmarks are sent to the classifier, and the sliding window is cleared except for the current frame. The sliding window is cleared because anything before the current frame would skew the Euclidean distance calculation. For example, a player is idle then performs a squat. If the sliding window is not cleared, the Euclidean distance calculation will continue to see the large difference in movement and will keep firing off landmarks to the classifier until the sliding window is filled with enough frames of squats. This contradicts the purpose of the movement detector, therefore, we clear the sliding window and begin to rebuild it again as seen in steps 3 and 4 in the diagram.

The size of the window, the number of averaged frames at the start and end of the sliding window, and the movement threshold are customizable. For example, on a Jetson Nano, we found the optimal window size to be 30, the start and end frames to be 5 each, and the threshold to be a Euclidean distance of 2.

E. Testing and Performance Evaluation Methodology

To identify which parts of the system needed to be op-timized and tuned, a more systematic way of measuring model performance needed to be established. The primary goal of optimization and tuning would be to increase F1 score metric performance, as minimising false categorizations while playing games was deemed the most important. The secondary goal was to reduce latency between exercises and inputs, but due to resource constraints, there was no formalization done for the measurement of the metric.

To quantify the system's F1 score performance, a majority of the architecture was blackboxed, minus the bluetooth device emulation, and the movement detection filter. The testing framework intakes entire unseen videos of exercises and classifies their exercise category. The predicted and actual classifications of the video frames are then compared to construct the F1 score.

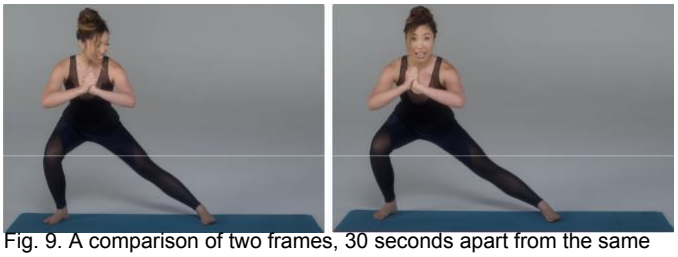

Fig. 9. A comparison of two frames, 30 seconds apart from the same right-lunge youtube demonstration video

Splitting test and train data by frames was considered, however it resulted in inaccurate quantifications of our perfor-mance. When splitting the data by shuffling all frames from all videos together, then splitting into a test set and a training set, frames from all videos would end up in both sets. Because of the nature of nearly all exercise videos to have more than a single repetition of each exercise, nearly identical frames frequently appear in the videos. To avoid overfitting by having nearly identical data in both the test set and training set, entire videos are processing during training and testing.

IV. Results

Evaluating our original model on unseen test data showed high F1, precision, and recall for every movement, with idle being slightly lower than the rest, as shown in Figure 10 and Figure 11. When the movement "left_oblique" was added, the scores remained similarly high (Fig. 12, Fig.13).

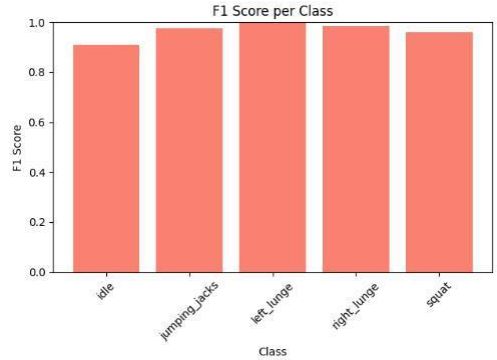

Fig. 10. F1 scores for the each of the movement labels: jumping jacks, left lunge, right lunge, squat, and idle.

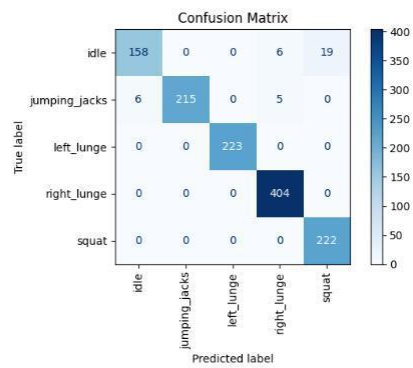

Fig. 11. Confusion matrix for the each of the movement labels: jumping jacks, left lunge, right lunge, squat, and idle.

Fig. 12. Confusion matrix for the each of the movement labels: jumping jacks, left lunge, right lunge, squat, idle, and left oblique.

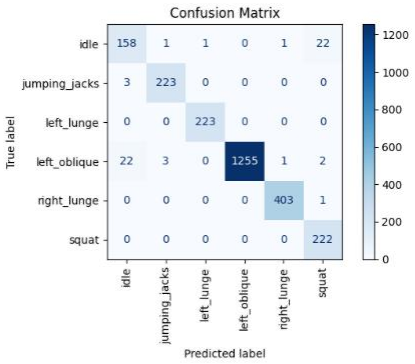

Fig. 13. Confusion matrix for the each of the movement labels: jumping jacks, left lunge, right lunge, squat, idle, and left oblique.

A. Discussion

We attribute a small part of the system's high F1 perfor-mance to the low similarity of the exercises we chose to test. Although they are common and likely to be chosen by users, there will likely be times when users would choose exercises that are even more similar to each other. Similarity of exercise classifications would directly impact the ability of the model to accurately classify images.

We believe the main contributor to the model's F1 perfor-mance is the use of BlazePose to extract the landmarks from images in vastly different environments. Having our model predict on landmarks meant it did not have to be tuned for differing clothes, camera angles, and other variables.

B. Insights

Throughout our research and implementation, we acquired many important insights about ML and Edge AI development. Firstly, we discovered how impactful dimensionality reduction is on efficiency of building and training a model. Using BlazePose to preprocess our data into landmarks eliminated an enormous amount of noise that usually hinders model accuracy and generalization. Secondly we initially assumed more layers and more epochs would be better for training Neural Networks. This turned out to be false, as our use of early stopping and dropout strategies on a smaller network ended up generalizing extremely well.

Another important insight we gained was the value of developing a model or solution with the target environment in mind. The Nvidia Jetson runs on a Linux-Based system and is configured with Python 3.6 and TensorFlow 2.5. Unfortu-nately, we had developed our model in Python environments using versions greater than 3.10, and TensorFlow 2.15. This led to compatibility issues when trying to deploy the model on the Nvidia Jetson. We spent countless hours trying to debug and find compatible library versions that our code needed to operate. In retrospect, if we had initially replicated our target environment from the start, we would have saved significant amounts of time, and this is something we would keep in mind moving forward. Removing capture bias led to greater generalization and accuracy of our model.

V. Conclusion

The goal of this paper was to provide a fun outlet for video game enthusiasts to stay active. While movement based gaming systems like the XBOX Kinect or VR headsets have tried to achieve this, the MovIn system expands the library to any game the user wants to play. MovIn uses computer vision, specifically a pose estimation model and a neural network, to classify the exercise the user is executing with outstanding accuracy. Then, the exercise is mapped to the corresponding HID key and executed in-game on the target device.

Blaze pose and joint angles definitions for exercises were helpful in streamlining the data acquisition process. Choosing to use Google's Blaze pose helped to reduce capture bias that we may have introduced with convolution. The extracted landmarks were then used to train the neural network. We then conducted model testing, using F1 score as the performance metric. We not only trained with our existing data, but we attempted to train off half a minute of our own footage.

This paper also discusses the application architecture that would be deployed on the Jetson Nano. This includes bluetooth integration, and application optimization with the movement detector and sliding window. Finally, we talk about the results, discussions and insights from model testing.

A. Future Work

An extension of this project would include a user-friendly graphical user interface (GUI) to enable users to interact with our application more easily, which would allow users to add new movements detected from their webcam and map them to specific keys. Another extension would be to further optimize the code to help reduce latency, making it possible to support more complex games.

References

[1] MulticoreWare, "Gesture Control Gaming," YouTube, May 14, 2020. https://www.youtube.com/watch?v=Q0794u64iIo (ac-cessed Dec. 06, 2024).

[2] "Introduction to Video Classification and Human Activity Recog-nition," Mar. 08, 2021. https://learnopencv.com/introduction-to-video-classification-and-human-activity-recognition/

[3] "On-device, Real-time Body Pose Tracking with MediaPipe BlazePose," research.google. https://research.google/blog/on-device-real-time-body-pose-tracking-with-mediapipe-blazepose/

[4] J. Arunnehru, G. Chamundeeswari, and S. Prasanna Bharathi, "Human action recognition using 3D convolutional neural networks with 3D motion cuboids in surveillance videos," Procedia Computer Science, vol. 133, pp. 471–477, 2018, doi: 10.1016/j.procs.2018.07.059.

[5] A. Gupta, K. Gupta, K. Gupta, and K. Gupta, "Human Activity Recog-nition Using Pose Estimation and Machine Learning Algorithm," in ISIC'21: International Semantic Intelligence Conference, New Delhi, India, Feb. 25-27, 2021. [Online]. Available:https://ceur-ws.org/Vol-2786/Paper40.pdf . [Accessed: Nov. 20, 2024].

[6] H. Kuehne, H. Jhuang, E. Garrote, T. Poggio, and T. Serre, "HMDB: A large video database for human motion recognition," in *2011 Inter-national Conference on Computer Vision (ICCV)*, Barcelona, Spain, 2011, pp. 2556-2563. doi: 10.1109/ICCV.2011.6126543.

[7] B. Baz, "Create a Bluetooth HID server," GitHub Gist, Oct. 17, 2024. [Online]. Available: https://gist.github.com/ukBaz/a47e71e7b87fbc851b27cde7d1c0fcf0. [Accessed: Dec. 1, 2024].